# Prelucrarea Imaginilor

Prof. Dr. Călin-Adrian POPA

## Cursul 6

3 Noiembrie 2025

# 6 Object detection

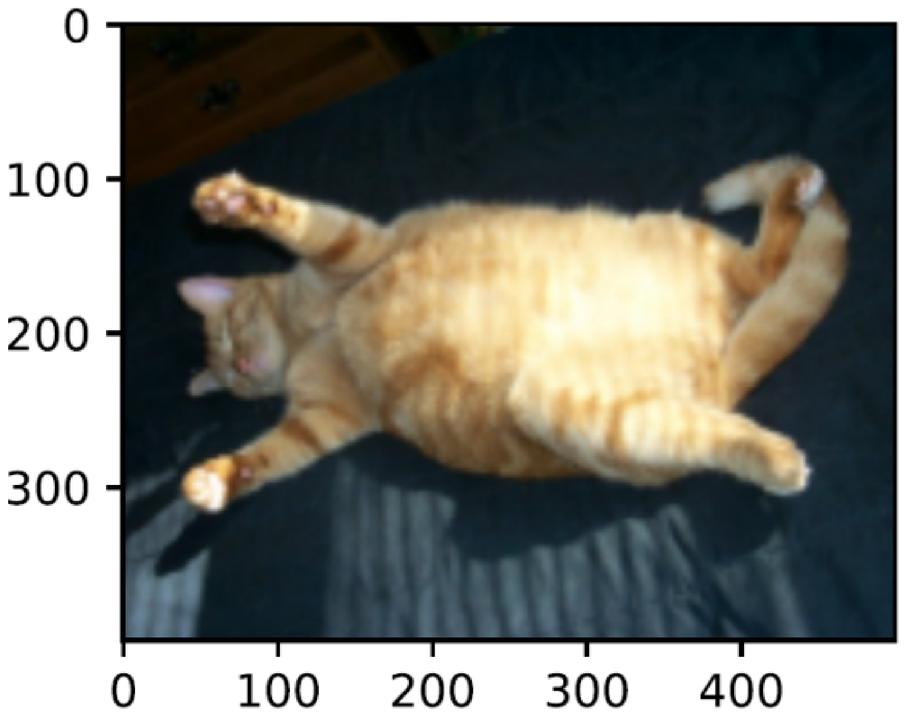
- in Chapter 3, we studied various convolutional neural networks that are commonly used in computer vision, and applied them to simple image classification tasks
- at the beginning of this chapter, we will describe two methods that may improve model generalization, namely *image augmentation* and *fine-tuning*
- since deep neural networks can effectively represent images in multiple levels, such layer-wise representations have been successfully used in various computer vision tasks, such as *object detection* and *semantic segmentation*
- following the key idea of leveraging layer-wise representations in computer vision, we will begin with major components and techniques for object detection

# 6.1 Image augmentation

- in Section 3.7, we mentioned that large datasets are a prerequisite for the success of deep neural networks in various applications
- *image augmentation* generates similar but distinct training examples after a series of random changes to the training images, thereby expanding the size of the training set
- alternatively, image augmentation can be motivated by the fact that random tweaks of training examples allow models to rely less on certain attributes, thereby improving their generalization ability
- for example, we can crop an image in different ways to make the object of interest appear in different positions, thereby reducing the dependence of a model on the position of the object
- we can also adjust factors such as brightness and color, to reduce a model's sensitivity to color
- it is probably true that image augmentation was indispensable for the success of AlexNet, at that time
- in this section, we will discuss this widely used technique in computer vision

# 6.1 Image augmentation

- in our investigation of common image augmentation methods, we will use the following $400 \times 500$ image as an example
- most image augmentation methods have a certain degree of randomness
- thus, to make it easier for us to observe the effect of image augmentation, we run the image augmentation method multiple times on the input image, and show all the results



# 6.1 Image augmentation

- *flipping* the image left and right usually does not change the category of the object
- this is one of the earliest and most widely used methods of image augmentation
- next, we flip the image left and right with a 50% chance



# 6.1 Image augmentation

- flipping up and down is not as common as flipping left and right
- but, at least for this example image, flipping up and down does not hinder recognition
- next, we flip the image up and down with a 50% chance



# 6.1 Image augmentation

- in the example image we used, the cat is in the middle of the image, but this may not be the case, in general
- in Section 3.5, we explained that the pooling layer can reduce the sensitivity of a convolutional layer to the target position
- in addition, we can also randomly *crop* the image to make objects appear in different positions in the image at different scales, which can also reduce the sensitivity of a model to the target position

# 6.1 Image augmentation

- in the example below, we randomly crop an area with an area of 10% $\sim$ 100% of the original area each time, and the ratio of width to height of this area is randomly selected from 0.5 $\sim$ 2
- then, the width and height of the region are both scaled to 200 pixels
- unless otherwise specified, the random number between $a$ and $b$ in this section refers to a continuous value obtained by random and uniform sampling from the interval $[a, b]$



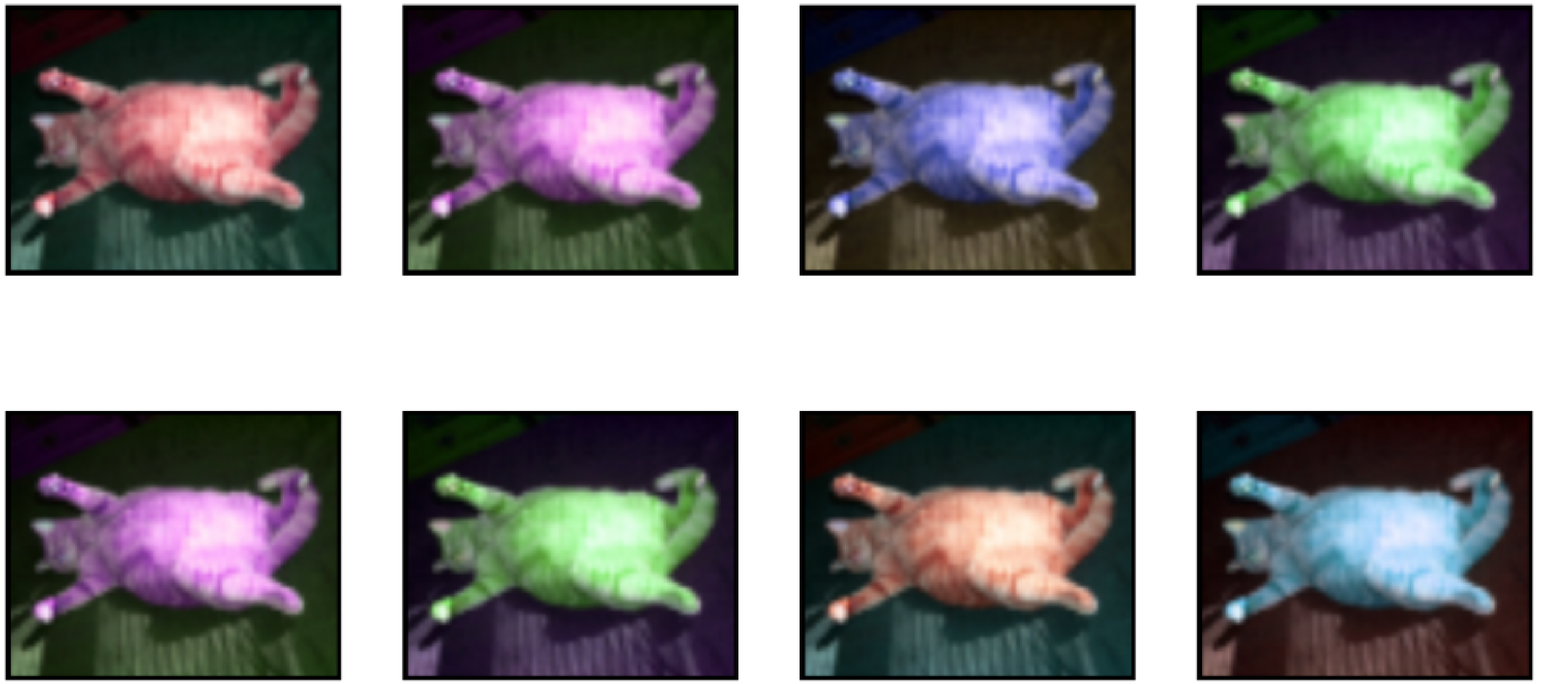
# 6.1 Image augmentation

- another augmentation method is *changing colors*
- we can change four aspects of the image color: brightness, contrast, saturation, and hue
- in the example below, we randomly change the brightness of the image to a value between 50% $(1 - 0.5)$ and 150% $(1 + 0.5)$ of the original image



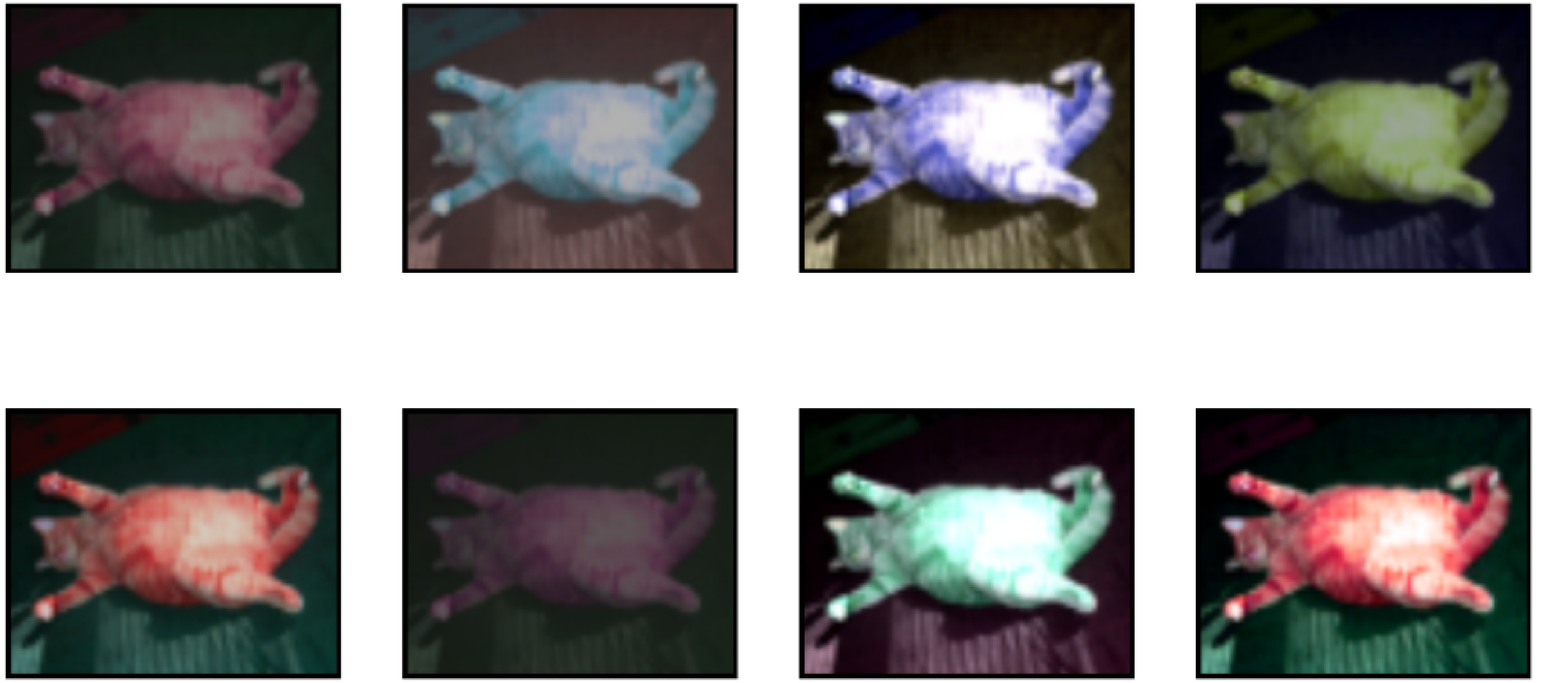
# 6.1 Image augmentation

- similarly, we can randomly change the hue of the image



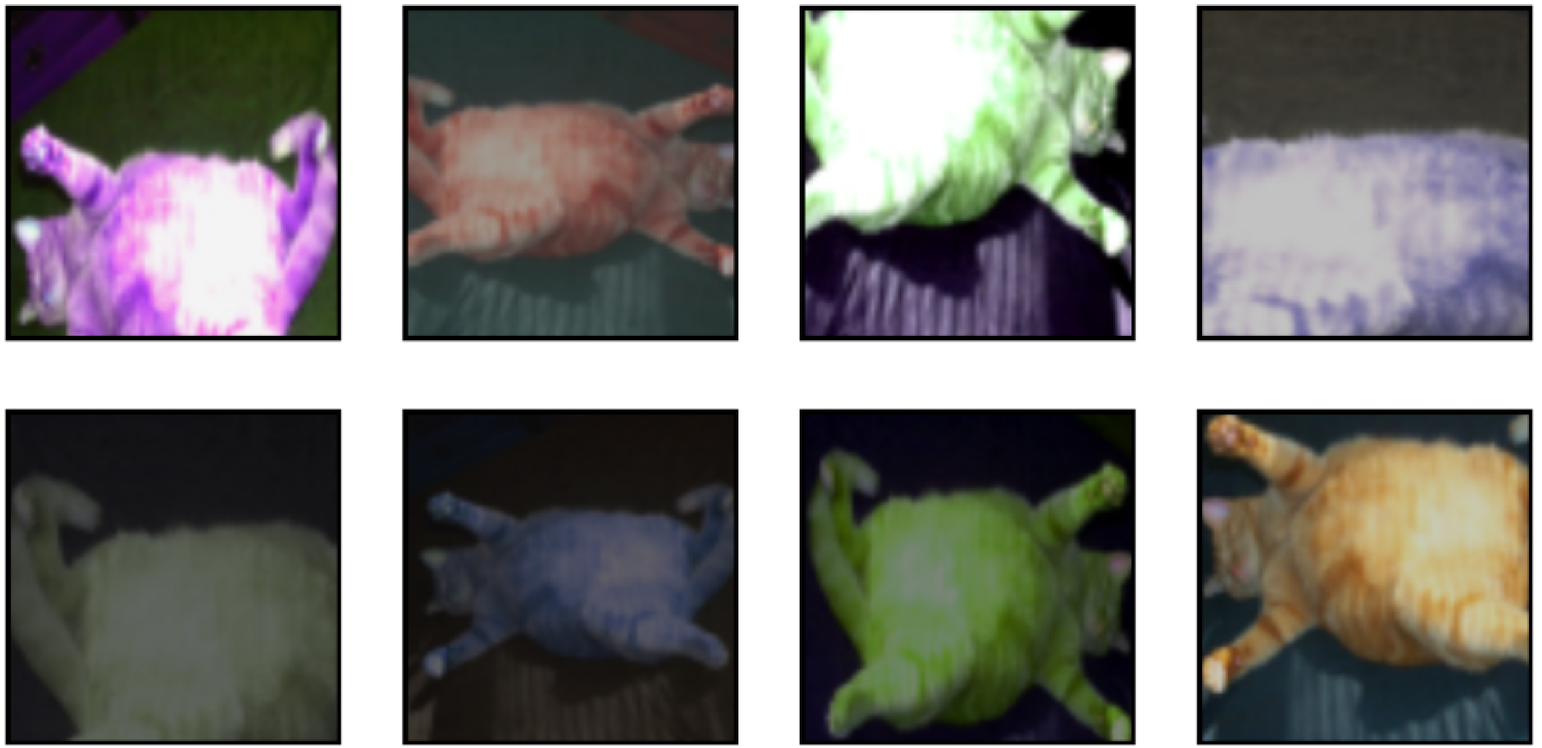
# 6.1 Image augmentation

- we can also randomly change the brightness, contrast, saturation, and hue of the image at the same time



# 6.1 Image augmentation

- in practice, we will combine multiple image augmentation methods
- for example, we can combine the different image augmentation methods defined above, and apply them to each image



- in order to obtain definitive results during prediction, we usually only apply image augmentation to training examples, and do not use image augmentation with random operations during prediction

# 6.2 Fine-tuning

- in earlier chapters, we discussed how to train models on the Fashion-MNIST training dataset, which has only 60000 images
- we also described ImageNet, the most widely used large-scale image dataset in academia, which has more than 10 million images and 1000 objects
- however, the size of the datasets that we usually encounter will be between those of the two datasets

# 6.2 Fine-tuning

- suppose that we want to recognize different types of chairs from images, and then recommend purchase links to users
- one possible method is to first identify 100 common chairs, take 1000 images of different angles for each chair, and then train a classification model on the collected image dataset
- although this chair dataset may be larger than the Fashion-MNIST dataset, the number of examples is still less than one-tenth of that in ImageNet
- this may lead to overfitting of complicated models that are suitable for ImageNet, on this chair dataset
- besides, due to the limited amount of training examples, the accuracy of the trained model may not meet practical requirements

# 6.2 Fine-tuning

- in order to address the above problems, an obvious solution is to collect more data
- however, collecting and labeling data can take a lot of time and money
- for example, in order to collect the ImageNet dataset, researchers have spent millions of dollars from research funding
- although the current data collection cost has been significantly reduced, this cost still cannot be ignored

# 6.2 Fine-tuning

- another solution is to apply *transfer learning* to transfer the knowledge learned from the *source dataset* to the *target dataset*
- for example, although most of the images in the ImageNet dataset have nothing to do with chairs, the model trained on this dataset may extract more general image features, which can help identify edges, textures, shapes, and object composition
- these similar features may also be effective for recognizing chairs

# 6.2 Fine-tuning

- in this section, we will introduce a common technique in transfer learning: *fine-tuning*
- as shown in Figure 1, fine-tuning consists of the following four steps:
  1. Pretrain a neural network model, i.e., the *source model*, on a source dataset (e.g., the ImageNet dataset).
  2. Create a new neural network model, i.e., the *target model*. This copies all model designs and their parameters of the source model, except the output layer. We assume that these model parameters contain the knowledge learned from the source dataset, and this knowledge will also be applicable to the target dataset. We also assume that the output layer of the source model is closely related to the labels of the source dataset; thus, it is not used in the target model.
  3. Add an output layer to the target model, whose number of outputs is the number of categories in the target dataset. Then randomly initialize the model parameters of this layer.
  4. Train the target model on the target dataset, such as a chair dataset. The output layer will be trained from scratch, while the parameters of all the other layers are fine-tuned based on the parameters of the source model.
- when target datasets are much smaller than source datasets, fine-tuning helps to improve models' generalization ability

# 6.2 Fine-tuning

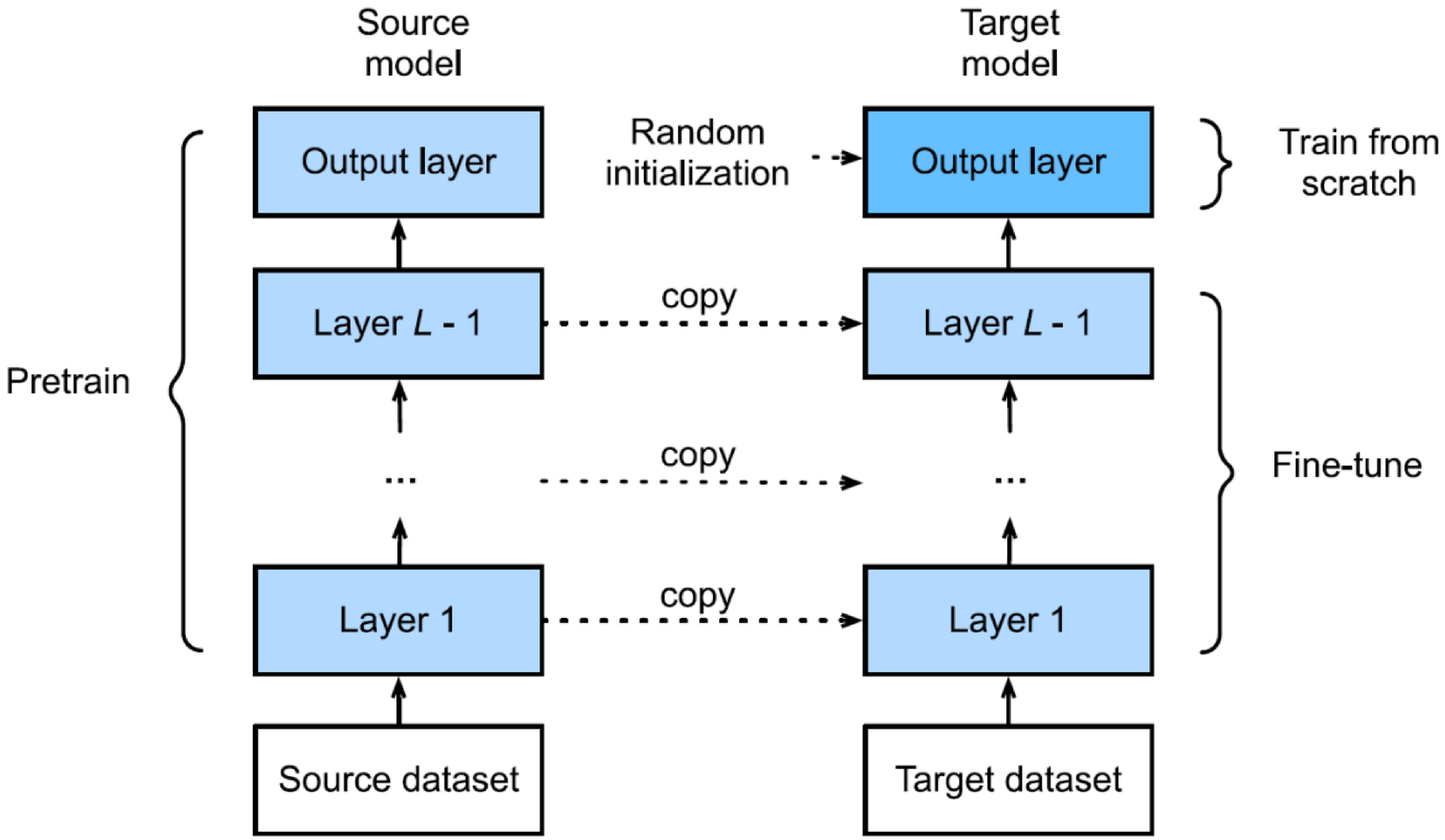



Figure 1: Fine-tuning.

# 6.3 Object detection and bounding boxes

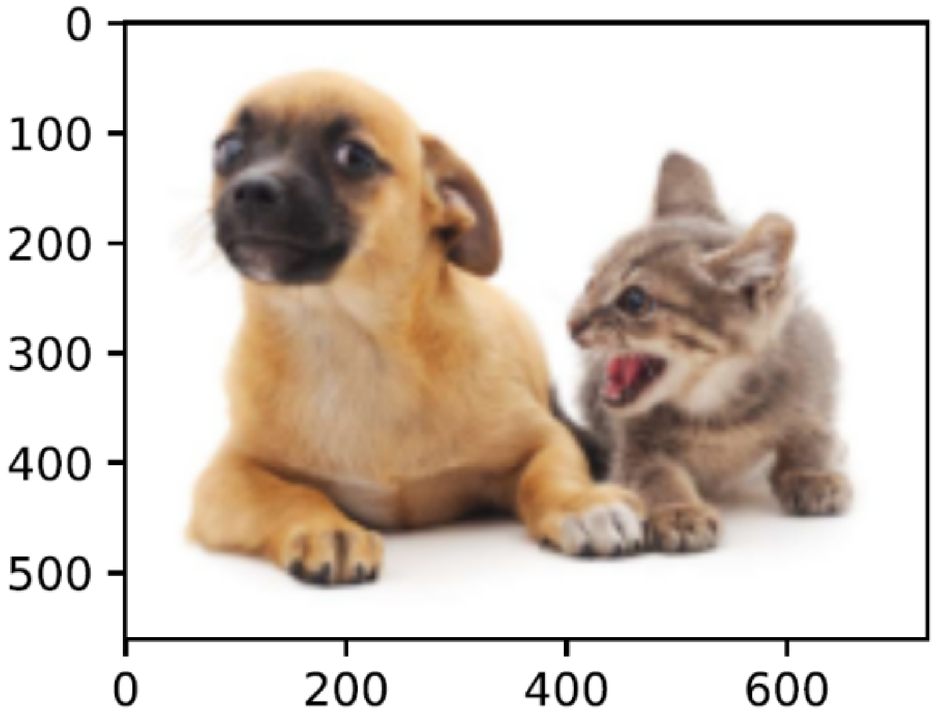
- in earlier sections (e.g., Section 3.7–Section 3.10), we introduced various models for image classification
- in image classification tasks, we assume that there is only *one* major object in the image, and we only focus on how to recognize its category
- however, there are often *multiple* objects in the image of interest
- we not only want to know their categories, but also their specific positions in the image
- in computer vision, we refer to such tasks as *object detection* (or *object recognition*)

# 6.3 Object detection and bounding boxes

- object detection has been widely applied in many fields
- for example, self-driving needs to plan traveling routes by detecting the positions of vehicles, pedestrians, roads, and obstacles in the captured video images
- besides, robots may use this technique to detect and localize objects of interest throughout their navigation of an environment
- moreover, security systems may need to detect abnormal objects, such as intruders or bombs

# 6.3 Object detection and bounding boxes

- in the next few sections, we will introduce several deep learning methods for object detection
- we will begin with an introduction to *positions* (or *locations*) of objects
- below we have the sample image to be used for object detection
- we can see that there is a dog on the left side of the image and a cat on the right; they are the two major objects in this image

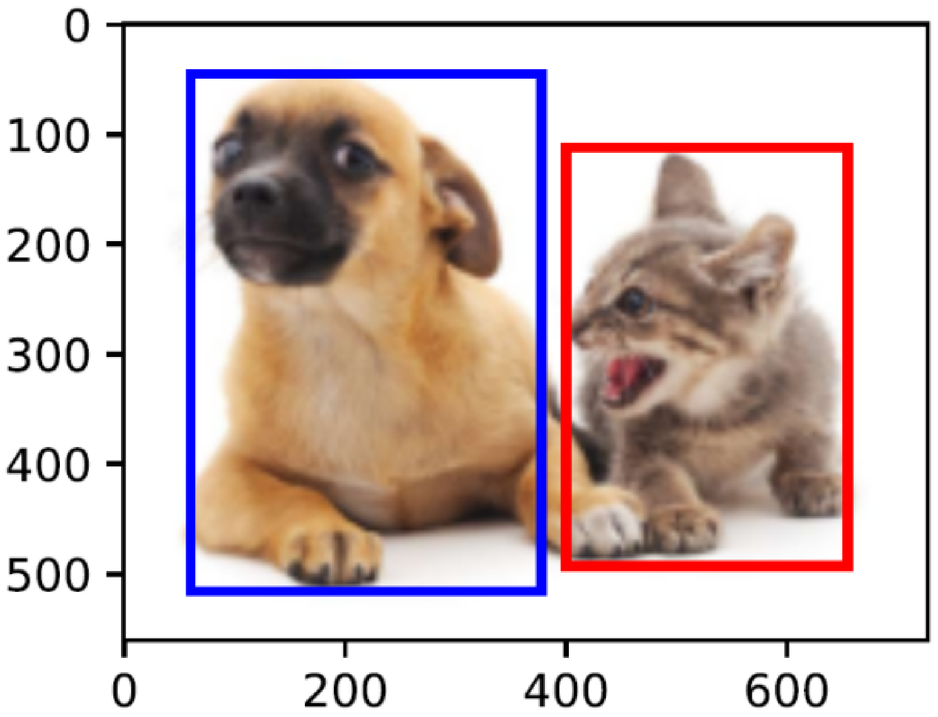


# 6.3 Object detection and bounding boxes

- in object detection, we usually use a *bounding box* to describe the spatial location of an object
- the bounding box is rectangular, which is determined by the $x$ and $y$ coordinates of the upper-left corner of the rectangle and the $x$ and $y$ coordinates of the lower-right corner
- another commonly used bounding box representation is the $(x, y)$-axis coordinates of the bounding box center, and the width and height of the box
- we will define the bounding boxes of the dog and the cat in the image based on the coordinate information
- the origin of the coordinates in the image is the upper-left corner of the image, and to the right and down are the positive directions of the $x$ and $y$ axes, respectively

# 6.3 Object detection and bounding boxes

- let us draw the bounding boxes in the image
- after adding the bounding boxes on the image, we can see that the main outlines of the two objects are basically inside the two boxes



# 6.4 Anchor boxes

- object detection algorithms usually sample a large number of regions in the input image, determine whether these regions contain objects of interest, and adjust the boundaries of the regions so as to predict the *ground-truth bounding boxes* of the objects more accurately
- different models may adopt different region sampling schemes
- here we introduce one of such methods: it generates multiple bounding boxes with varying scales and aspect ratios, centered on each pixel
- these bounding boxes are called *anchor boxes*
- we will design an object detection model based on anchor boxes in Section 6.6

# 6.4 Anchor boxes

- suppose that the input image has a height of $h$ and a width of $w$
- we generate anchor boxes with different shapes centered on each pixel of the image
- let the *scale* be $s \in (0, 1]$ and the *aspect ratio* (ratio of width to height) be $r > 0$
- then, the width and height of the anchor box are $ws\sqrt{r}$ and $hs/\sqrt{r}$, respectively
- note that when the center position is given, an anchor box with known width and height is determined

# 6.4 Anchor boxes

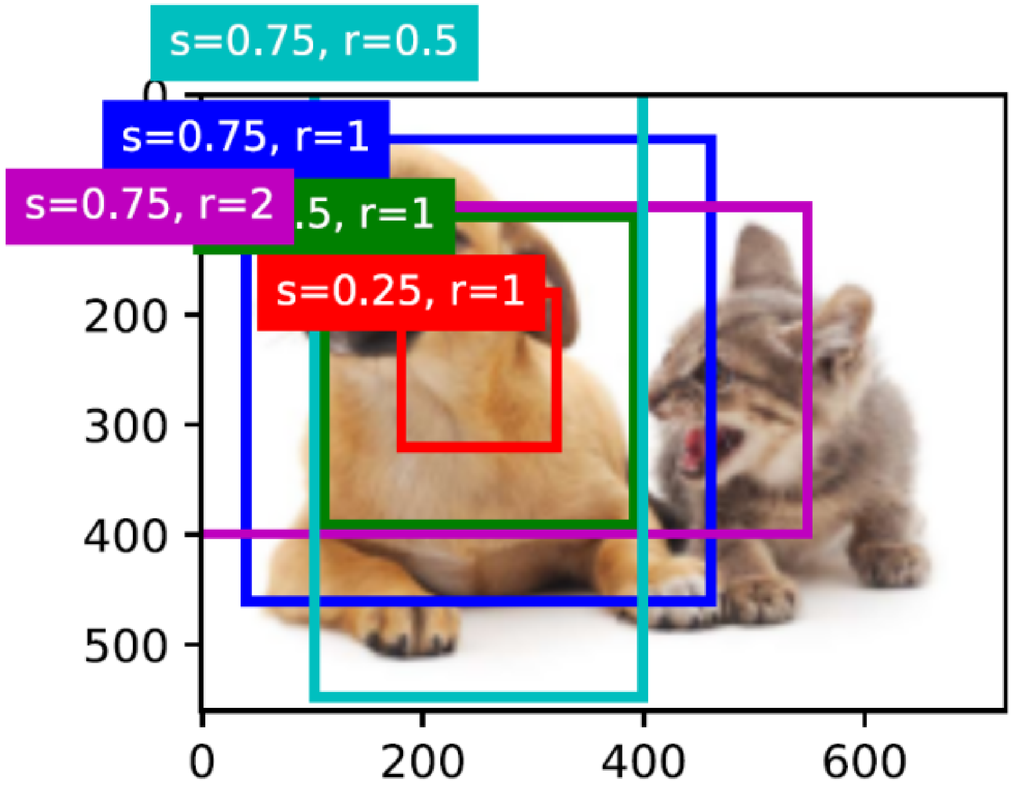
- to generate multiple anchor boxes with different shapes, let us set a series of scales $s_1, \ldots, s_n$ and a series of aspect ratios $r_1, \ldots, r_m$
- when using all the combinations of these scales and aspect ratios with each pixel as the center, the input image will have a total of $whnm$ anchor boxes
- although these anchor boxes may cover all the ground-truth bounding boxes, the computational complexity is generally too high
- in practice, we can only consider those combinations containing $s_1$ or $r_1$:

$$(s_1, r_1), (s_1, r_2), \ldots, (s_1, r_m), (s_2, r_1), (s_3, r_1), \ldots, (s_n, r_1).$$

- that is to say, the number of anchor boxes centered on the same pixel is $n + m - 1$
- for the entire input image, we will generate a total of $wh(n + m - 1)$ anchor boxes

# 6.4 Anchor boxes

- for our example, we can draw all the anchor boxes centered on $(250, 250)$ in the image
- as we can see, the blue anchor box with a scale of 0.75 and an aspect ratio of 1 well surrounds the dog in the image



# 6.4 Anchor boxes

- we just mentioned that an anchor box “well” surrounds the dog in the image
- if the ground-truth bounding box of the object is known, how can “well” here be quantified?
- intuitively, we can measure the similarity between the anchor box and the ground-truth bounding box
- we know that the *Jaccard index* can measure the similarity between two sets
- given sets $\mathcal{A}$ and $\mathcal{B}$, their Jaccard index is the size of their intersection divided by the size of their union:

$$J(\mathcal{A}, \mathcal{B}) = \frac{|\mathcal{A} \cap \mathcal{B}|}{|\mathcal{A} \cup \mathcal{B}|}.$$

# 6.4 Anchor boxes

- in fact, we can consider the pixel area of any bounding box as a set of pixels
- in this way, we can measure the similarity of the two bounding boxes by the Jaccard index of their pixel sets
- for two bounding boxes, we usually refer to their Jaccard index as the *intersection over union* (*IoU*), which is the ratio of their intersection area to their union area, as shown in Figure 2
- the range of an IoU is between 0 and 1: 0 means that two bounding boxes do not overlap at all, while 1 indicates that the two bounding boxes are equal

# 6.4 Anchor boxes

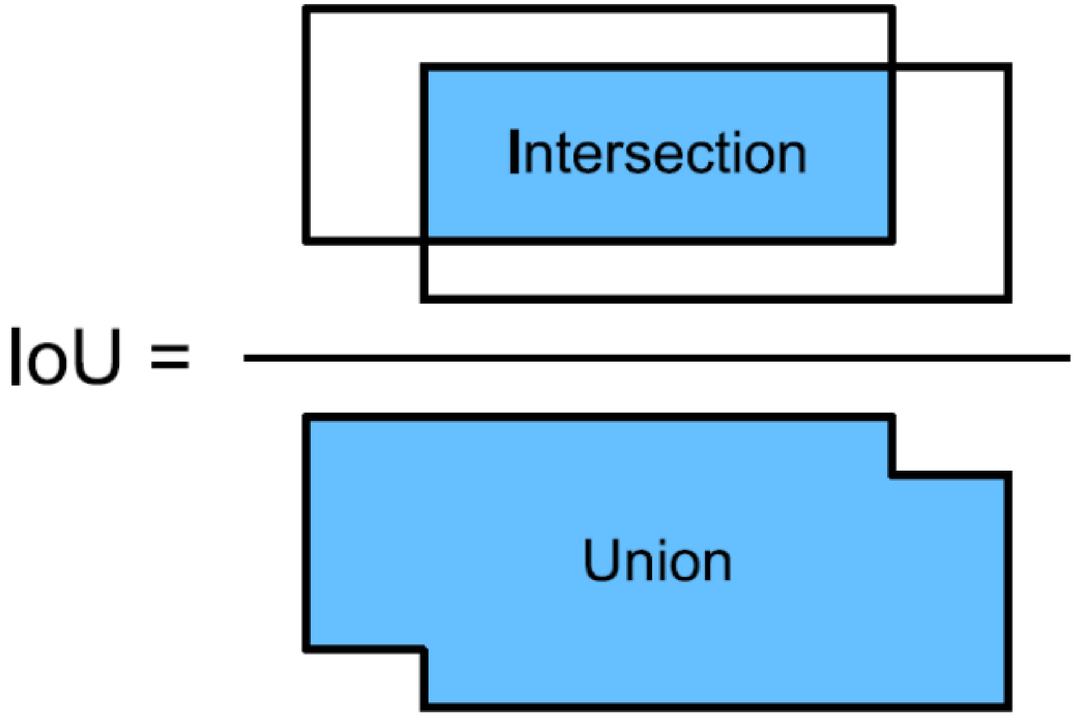



Figure 2: IoU is the ratio of the intersection area to the union area of two bounding boxes.

# 6.4 Anchor boxes

- we will use IoU to measure the similarity between anchor boxes and ground-truth bounding boxes, and between different anchor boxes
- in a training dataset, we consider each anchor box as a training example
- in order to train an object detection model, we need *class* and *offset* labels for each anchor box, where the former is the class of the object relevant to the anchor box, and the latter is the offset of the ground-truth bounding box relative to the anchor box
- during prediction, for each image, we generate multiple anchor boxes, predict classes and offsets for all the anchor boxes, adjust their positions according to the predicted offsets to obtain the predicted bounding boxes, and finally only output those predicted bounding boxes that satisfy certain criteria

# 6.4 Anchor boxes

- in general, an object detection training set comes with labels for locations of *ground-truth bounding boxes* and classes of their surrounded objects
- to label any generated *anchor box*, we refer to the labeled location and class of its *assigned* ground-truth bounding box that is closest to the anchor box
- in the following, we describe an algorithm for assigning closest ground-truth bounding boxes to anchor boxes
- given an image, suppose that the anchor boxes are $A_1, A_2, \ldots, A_{n_a}$, and the ground-truth bounding boxes are $B_1, B_2, \ldots, B_{n_b}$, where $n_a \geq n_b$
- let us define a matrix $\boldsymbol{X} \in \mathbb{R}^{n_a \times n_b}$, whose element $x_{ij}$ in the $i$th row and $j$th column is the IoU of the anchor box $A_i$ and the ground-truth bounding box $B_j$

# 6.4 Anchor boxes

- the algorithm consists of the following steps:
  1. Find the largest element in matrix $\boldsymbol{X}$, and denote its row and column indices as $i_1$ and $j_1$, respectively. Then, the ground-truth bounding box $B_{j_1}$ is assigned to the anchor box $A_{i_1}$. This is quite intuitive, because $A_{i_1}$ and $B_{j_1}$ are the closest among all the pairs of anchor boxes and ground-truth bounding boxes. After the first assignment, discard all the elements in the $i_1$th row and the $j_1$th column in matrix $\boldsymbol{X}$.
  2. Find the largest of the remaining elements in matrix $\boldsymbol{X}$, and denote its row and column indices as $i_2$ and $j_2$, respectively. We assign ground-truth bounding box $B_{j_2}$ to anchor box $A_{i_2}$, and discard all the elements in the $i_2$th row and the $j_2$th column in matrix $\boldsymbol{X}$.
  3. At this point, elements in two rows and two columns in matrix $\boldsymbol{X}$ have been discarded. We proceed until all elements in $n_b$ columns in matrix $\boldsymbol{X}$ are discarded. At this time, we have assigned a ground-truth bounding box to each of $n_b$ anchor boxes.
  4. Only traverse through the remaining $n_a - n_b$ anchor boxes. For example, given any anchor box $A_i$, find the ground-truth bounding box $B_j$ with the largest IoU with $A_i$ throughout the $i$th row of matrix $\boldsymbol{X}$, and assign $B_j$ to $A_i$ only if this IoU is greater than a predefined threshold.

# 6.4 Anchor boxes

- let us illustrate the above algorithm using a concrete example
- as shown in Figure 3 (left), assuming that the maximum value in matrix $\boldsymbol{X}$ is $x_{23}$, we assign the ground-truth bounding box $B_3$ to the anchor box $A_2$
- then, we discard all the elements in row 2 and column 3 of the matrix, find the largest $x_{71}$ in the remaining elements (shaded area), and assign the ground-truth bounding box $B_1$ to the anchor box $A_7$
- next, as shown in Figure 3 (middle), discard all the elements in row 7 and column 1 of the matrix, find the largest $x_{54}$ in the remaining elements (shaded area), and assign the ground-truth bounding box $B_4$ to the anchor box $A_5$
- finally, as shown in Figure 3 (right), discard all the elements in row 5 and column 4 of the matrix, find the largest $x_{92}$ in the remaining elements (shaded area), and assign the ground-truth bounding box $B_2$ to the anchor box $A_9$
- after that, we only need to traverse through the remaining anchor boxes $A_1, A_3, A_4, A_6, A_8$ and determine whether to assign them ground-truth bounding boxes according to the threshold

# 6.4 Anchor boxes

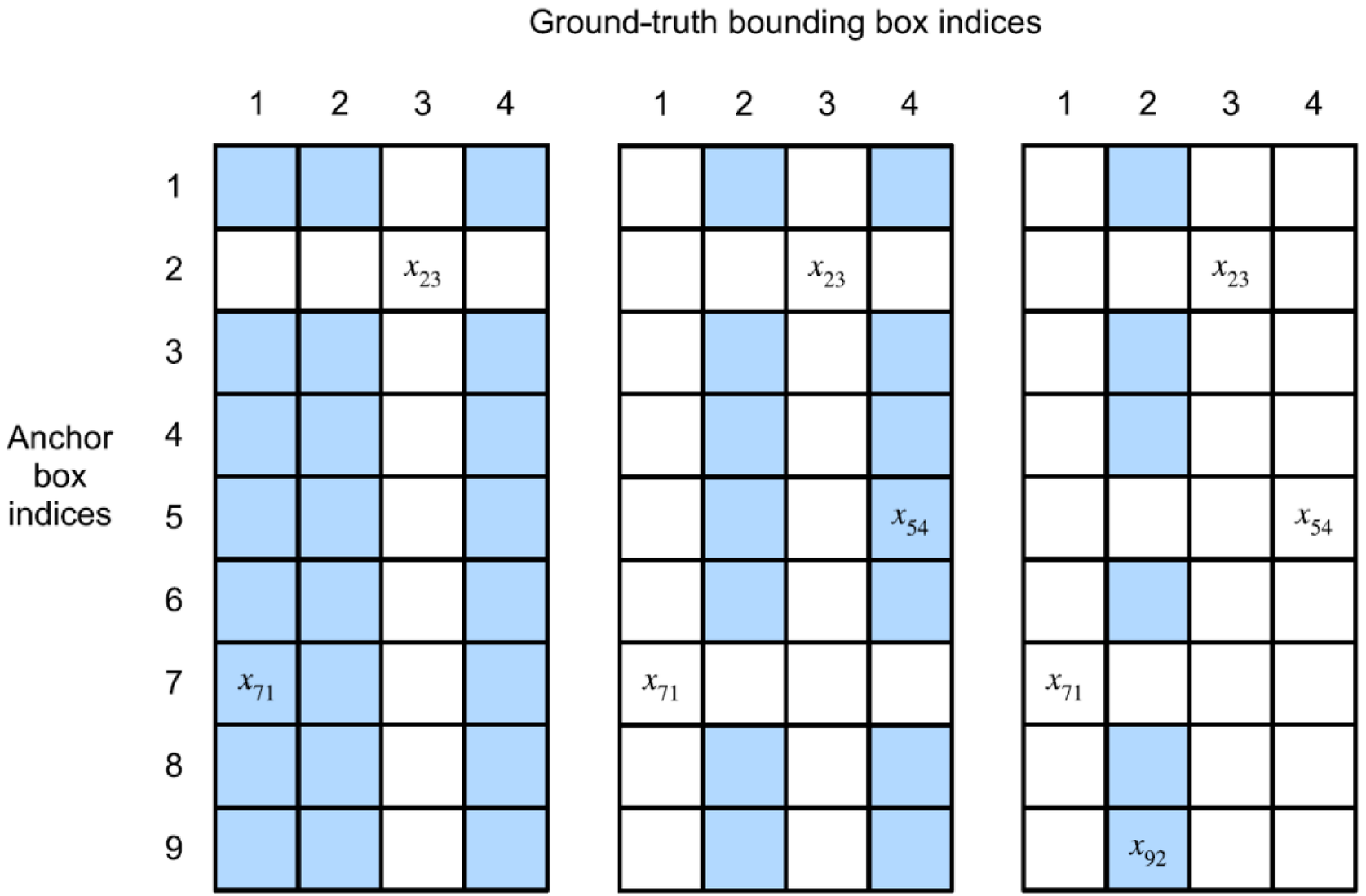



Figure 3: Assigning ground-truth bounding boxes to anchor boxes.

# 6.4 Anchor boxes

- now, we can label the class and offset for each anchor box
- suppose that an anchor box $A$ is assigned a ground-truth bounding box $B$
- on one hand, the class of the anchor box $A$ will be labeled as that of $B$
- on the other hand, the offset of the anchor box $A$ will be labeled according to the relative position between the central coordinates of $B$ and $A$, together with the relative size between these two boxes
- given varying positions and sizes of different boxes in the dataset, we can apply transformations to those relative positions and sizes that may lead to more uniformly distributed offsets that are easier to fit
- here, we describe a common transformation; given the central coordinates of $A$ and $B$ as $(x_a, y_a)$ and $(x_b, y_b)$, their widths as $w_a$ and $w_b$, and their heights as $h_a$ and $h_b$, respectively
- we may label the offset of $A$ as:

$$\left( \frac{\frac{x_b - x_a}{w_a} - \mu_x}{\sigma_x}, \frac{\frac{y_b - y_a}{h_a} - \mu_y}{\sigma_y}, \frac{\log \frac{w_b}{w_a} - \mu_w}{\sigma_w}, \frac{\log \frac{h_b}{h_a} - \mu_h}{\sigma_h} \right),$$

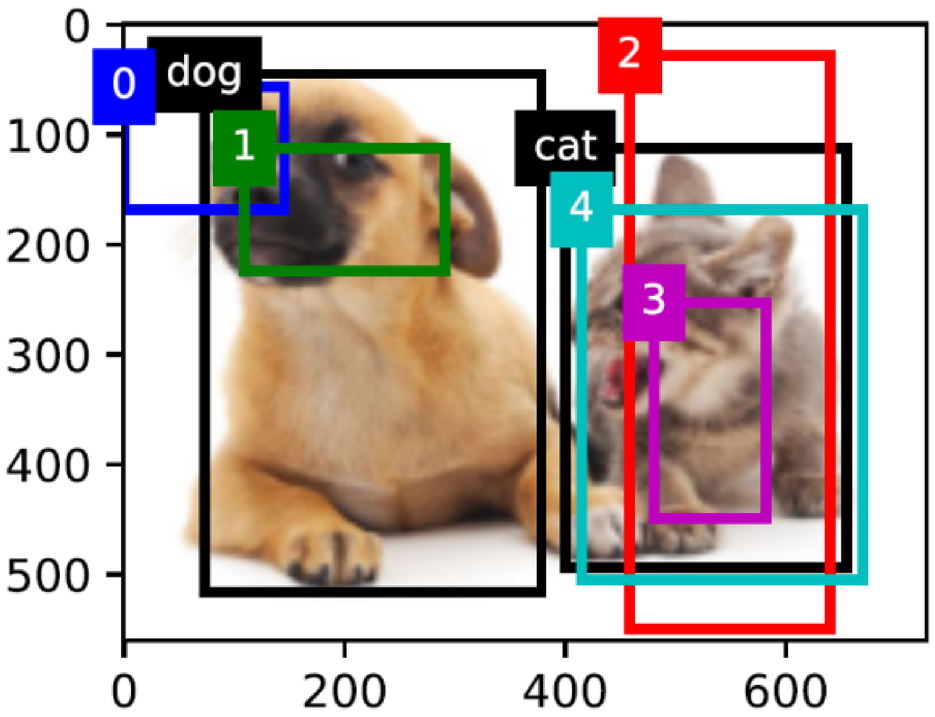
where the default values of the constants are: $\mu_x = \mu_y = \mu_w = \mu_h = 0$, $\sigma_x = \sigma_y = 0.1$, and $\sigma_w = \sigma_h = 0.2$

# 6.4 Anchor boxes

- if an anchor box is not assigned a ground-truth bounding box, we just label the class of the anchor box as “background”
- anchor boxes whose classes are background are often referred to as *negative* anchor boxes, and the rest are called *positive* anchor boxes
- let us illustrate anchor box labeling via a concrete example
- we define ground-truth bounding boxes for the dog and cat in the loaded image, where the first element is the class (1 for dog and 2 for cat) and the remaining four elements are the $(x, y)$-axis coordinates at the upper-left corner and the lower-right corner

# 6.4 Anchor boxes

- we also construct five anchor boxes to be labeled using the coordinates of the upper-left corner and the lower-right corner: $A_0, \ldots, A_4$ (the index starts from 0)
- then, we plot these ground-truth bounding boxes and anchor boxes in the image



# 6.4 Anchor boxes

- we can label classes and offsets of these anchor boxes based on the ground-truth bounding boxes for the dog and cat
- in this example, indices of the background, dog, and cat classes are 0, 1, and 2, respectively
- let us analyze the class labels that will be assigned based on anchor box and ground-truth bounding box positions in the image
- first, among all the pairs of anchor boxes and ground-truth bounding boxes, the IoU of the anchor box $A_4$ and the ground-truth bounding box of the cat is the largest
- thus, the class of $A_4$ is labeled as the cat

# 6.4 Anchor boxes

- taking out pairs containing $A_4$ or the ground-truth bounding box of the cat, among the rest, the pair of the anchor box $A_1$ and the ground-truth bounding box of the dog has the largest IoU
- so the class of $A_1$ is labeled as the dog
- next, we need to traverse through the remaining three unlabeled anchor boxes: $A_0$, $A_2$, and $A_3$
- for $A_0$, the class of the ground-truth bounding box with the largest IoU is the dog, but the IoU is below the predefined threshold (0.5), so the class is labeled as background
- for $A_2$, the class of the ground-truth bounding box with the largest IoU is the cat and the IoU exceeds the threshold, so the class is labeled as the cat
- for $A_3$, the class of the ground-truth bounding box with the largest IoU is the cat, but the value is below the threshold, so the class is labeled as background

# 6.4 Anchor boxes

- since we do not care about background detection, offsets of the negative class should not affect the objective function
- through element-wise multiplications, zeros in a mask variable will filter out negative class offsets before calculating the objective function
- during prediction, we generate multiple anchor boxes for the image, and predict classes and offsets for each of them
- a *predicted bounding box* is thus obtained according to an anchor box, with its predicted offset
- when there are many anchor boxes, many similar (with significant overlap) predicted bounding boxes can be potentially output for surrounding the same object
- to simplify the output, we can merge similar predicted bounding boxes that belong to the same object by using *non-maximum suppression* (NMS)

# 6.4 Anchor boxes

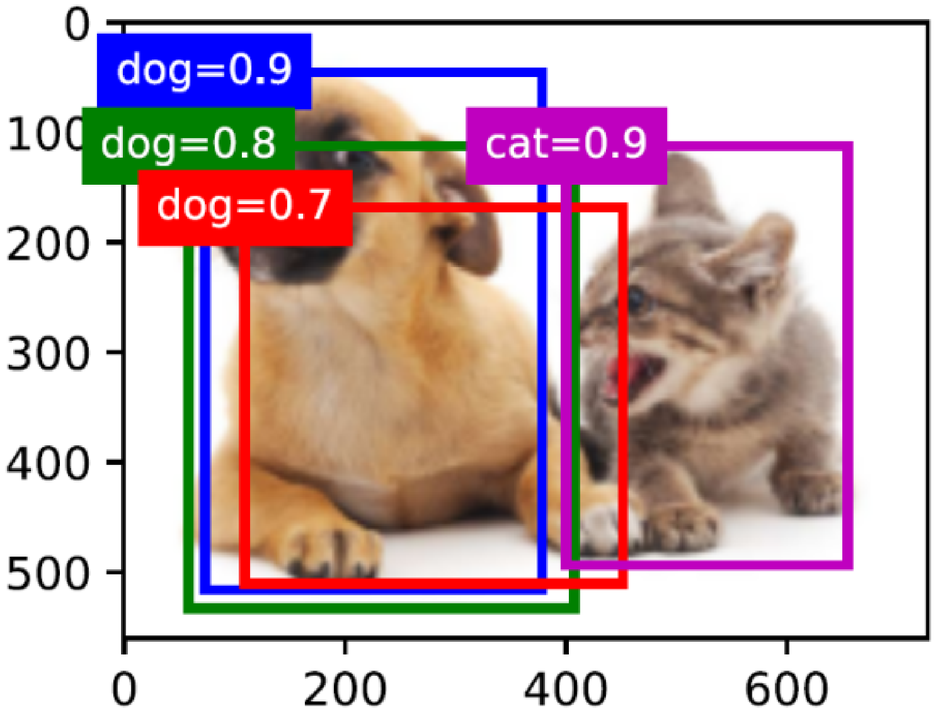
- here is how non-maximum suppression works
- for a predicted bounding box $B$, the object detection model calculates the predicted likelihood for each class
- denoting by $p$ the largest predicted likelihood, the class corresponding to this probability is the predicted class for $B$
- specifically, we refer to $p$ as the *confidence* (score) of the predicted bounding box $B$
- on the same image, all the predicted non-background bounding boxes are sorted by confidence, in descending order, to generate a list $L$

# 6.4 Anchor boxes

- then, we manipulate the sorted list $L$ in the following steps:
  1. Select the predicted bounding box $B_1$ with the highest confidence from $L$ as a basis, and remove all non-basis predicted bounding boxes whose IoU with $B_1$ exceeds a predefined threshold $\epsilon$ from $L$. At this point, $L$ keeps the predicted bounding box with the highest confidence, but drops others that are too similar to it. In a nutshell, those with *non-maximum* confidence scores are *suppressed*.
  2. Select the predicted bounding box $B_2$ with the second highest confidence from $L$ as another basis, and remove from $L$ all non-basis predicted bounding boxes whose IoU with $B_2$ exceeds $\epsilon$.
  3. Repeat the above process until all the predicted bounding boxes in $L$ have been used as a basis. At this time, the IoU of any pair of predicted bounding boxes in $L$ is below the threshold $\epsilon$; thus, no pair is too similar with each other.
  4. Output all the predicted bounding boxes in the list $L$.

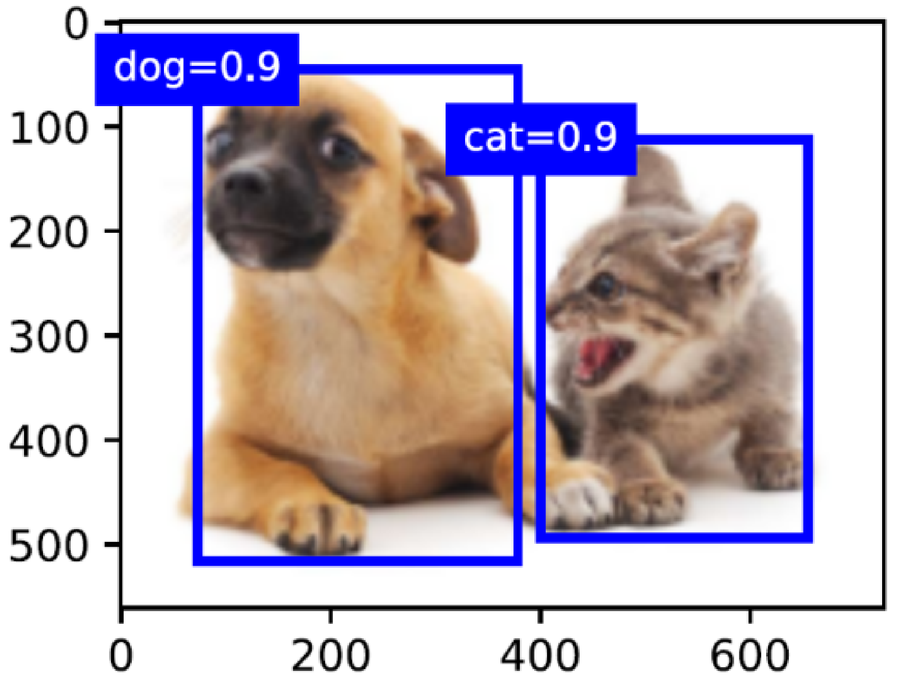
# 6.4 Anchor boxes

- now, let us apply the above methods to a concrete example with four anchor boxes
- for simplicity, we assume that the predicted offsets are all zeros
- this means that the predicted bounding boxes are anchor boxes
- for each class among the background, dog, and cat, we also define its predicted likelihood
- we can plot these predicted bounding boxes with their confidence on the image



# 6.4 Anchor boxes

- now, we can perform non-maximum suppression, where the threshold is set to 0.5
- after removing those predicted bounding boxes of the background class, we can output the final predicted bounding box kept by non-maximum suppression




- in practice, we can remove predicted bounding boxes with lower confidence even before performing non-maximum suppression, thereby reducing computation in this algorithm
- we may also post-process the output of non-maximum suppression, for example, by only keeping results with higher confidence in the final output

# 6.5 Multiscale object detection

- in Section 6.4, we described a method to generate multiple anchor boxes centered on each pixel of an input image
- essentially, these anchor boxes represent samples of different regions of the image
- however, we may end up with too many anchor boxes to compute, if they are generated for *every* pixel
- for example, for a $561 \times 728$ input image, if five anchor boxes with varying shapes are generated for each pixel as their center, over two million anchor boxes ($561 \times 728 \times 5$) need to be labeled and predicted on the image

# 6.5 Multiscale object detection

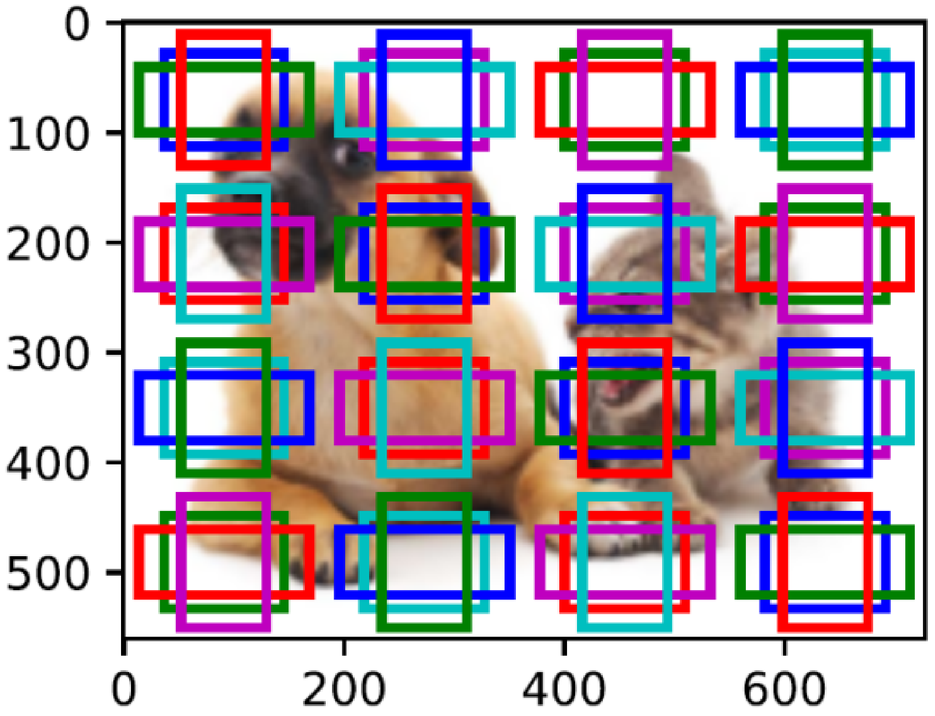
- it is not difficult to reduce anchor boxes on an image
- for instance, we can just uniformly sample a small portion of pixels from the input image to generate anchor boxes centered on them
- in addition, at different scales, we can generate different numbers of anchor boxes of different sizes
- intuitively, smaller objects are more likely to appear on an image than larger ones
- as an example, $1 \times 1$, $1 \times 2$, and $2 \times 2$ objects can appear on a $2 \times 2$ image in 4, 2, and 1 possible ways, respectively
- therefore, when using smaller anchor boxes to detect smaller objects, we can sample more regions, while for larger objects we can sample fewer regions

# 6.5 Multiscale object detection

- recall that, in Section 3.2, we call a two-dimensional array output of a convolutional layer a feature map
- by defining the feature map shape, we can determine centers of uniformly sampled anchor boxes on any image
- we generate anchor boxes on the feature map with each unit (pixel) as the anchor box center
- since centers of the anchor boxes are spread over all units on the feature map, these centers must be *uniformly* distributed on any input image in terms of their relative spatial positions
- more concretely, given the width and height of the feature map $fmap_w$ and $fmap_h$, respectively, we will uniformly sample pixels in $fmap_h$ rows and $fmap_w$ columns, on any input image
- centered on these uniformly sampled pixels, anchor boxes of scale $s$ and different aspect ratios will be generated

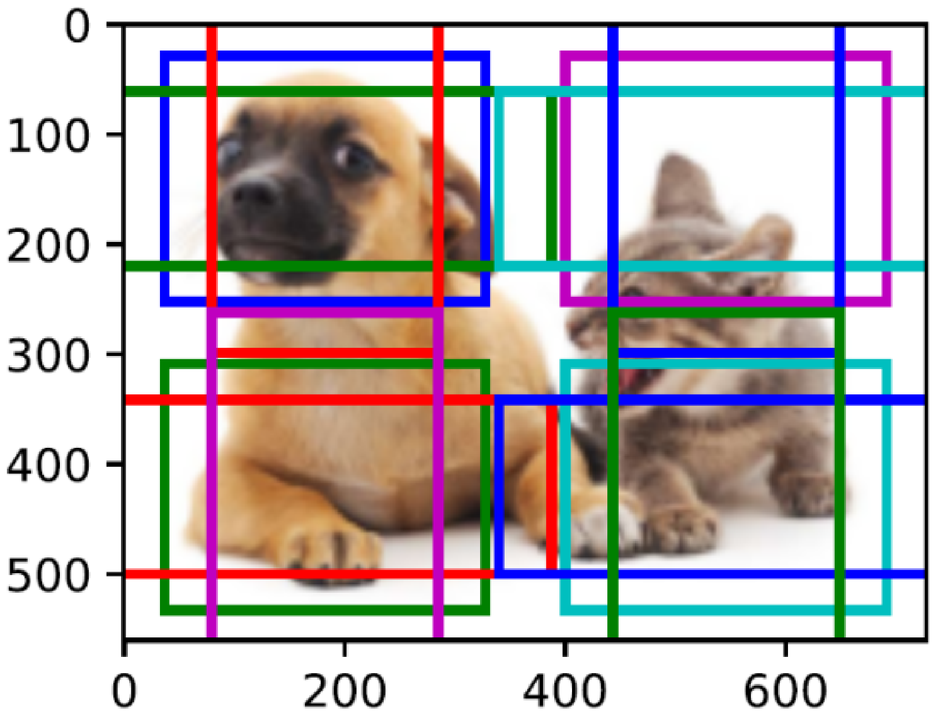
# 6.5 Multiscale object detection

- first, let us consider detection of small objects
- in order to make it easier to distinguish when displayed, the anchor boxes with different centers here do not overlap: the anchor box scale is set to 0.15, and the height and width of the feature map are set to 4
- we can see that the centers of the anchor boxes in 4 rows and 4 columns on the image are uniformly distributed



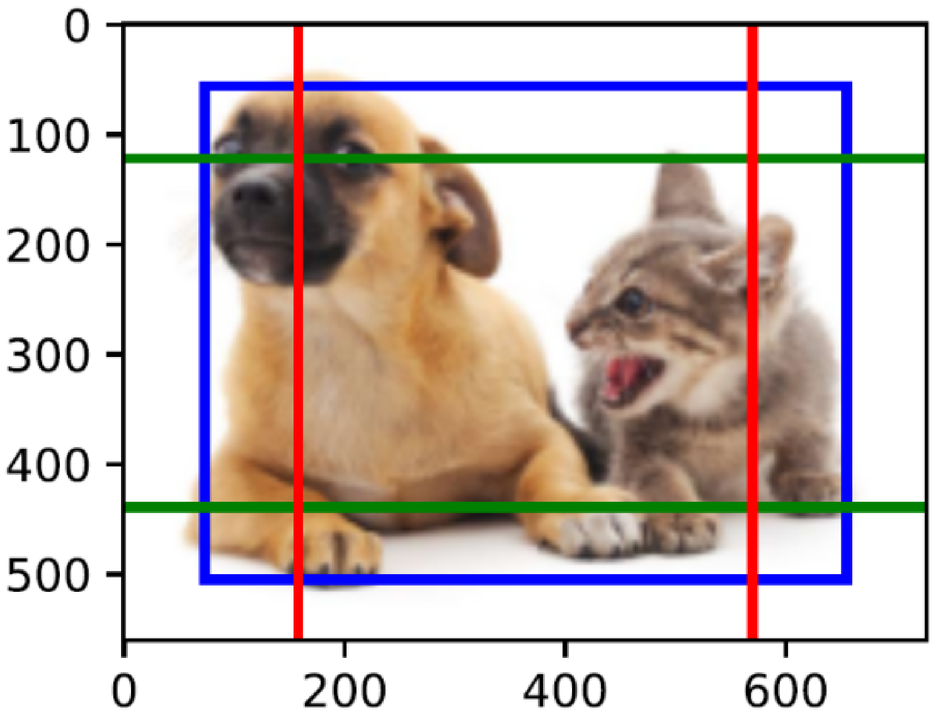
# 6.5 Multiscale object detection

- we move on to reduce the height and width of the feature map by half, and use larger anchor boxes to detect larger objects
- when the scale is set to 0.4, some anchor boxes will overlap with each other



# 6.5 Multiscale object detection

- finally, we further reduce the height and width of the feature map by half, and increase the anchor box scale to 0.8
- now, the center of the anchor box is the center of the image



# 6.5 Multiscale object detection

- since we have generated multiscale anchor boxes, we will use them to detect objects of various sizes at different scales
- in the following, we introduce a CNN-based multiscale object detection method that we will detail in Section 6.6
- at some scale, say that we have $c$ feature maps of shape $h \times w$
- using the method described above, we generate $hw$ sets of anchor boxes, where each set has $a$ anchor boxes with the same center
- for example, at the first scale in the examples above, given ten (number of channels) $4 \times 4$ feature maps, we generate 16 sets of anchor boxes, where each set contains 3 anchor boxes with the same center
- next, each anchor box is labeled with the class and offset based on ground-truth bounding boxes
- at the current scale, the object detection model needs to predict the classes and offsets of $hw$ sets of anchor boxes on the input image, where different sets have different centers

# 6.5 Multiscale object detection

- assume that the $c$ feature maps here are the intermediate outputs obtained by the CNN forward propagation based on the input image
- since there are $hw$ different spatial positions on each feature map, the same spatial position can be thought of as having $c$ units
- according to the definition of the receptive field in Section 3.2, these $c$ units at the same spatial position of the feature maps have the same receptive field on the input image: they represent the input image information in the same receptive field
- therefore, we can transform the $c$ units of the feature maps at the same spatial position into the classes and offsets of the $a$ anchor boxes generated using this spatial position
- in essence, we use the information of the input image in a certain receptive field to predict the classes and offsets of the anchor boxes that are close to that receptive field on the input image

# 6.5 Multiscale object detection

- when the feature maps at different layers have varying-size receptive fields on the input image, they can be used to detect objects of different sizes
- for example, we can design a neural network where units of feature maps that are closer to the output layer have wider receptive fields, so they can detect larger objects from the input image
- in a nutshell, we can leverage layer-wise representations of images at multiple levels by deep neural networks for multiscale object detection
- we will show how this works through a concrete example in Section 6.6

# 6.6 Single shot multibox detection

- in Section 6.3–Section 6.5, we introduced bounding boxes, anchor boxes, and multiscale object detection
- now, we are ready to use such background knowledge to design an object detection model: *single shot multibox detection* (*SSD*)
- this model is simple, fast, and widely used
- although this is just one of vast amounts of object detection models, some of the design principles and implementation details in this section are also applicable to other models

# 6.6 Single shot multibox detection

- Figure 4 provides an overview of the design of single-shot multibox detection
- this model mainly consists of a base network, followed by several multiscale feature map blocks
- the base network is for extracting features from the input image, so it can use a deep CNN
- for example, the original single-shot multibox detection paper adopts a VGG network truncated before the classification layer, while ResNet has also been commonly used
- through our design, we can make the base network output larger feature maps, so as to generate more anchor boxes for detecting smaller objects
- subsequently, each multiscale feature map block reduces (e.g., by half) the height and width of the feature maps from the previous block, and enables each unit of the feature maps to increase its receptive field on the input image

# 6.6 Single shot multibox detection

- recall the design of multiscale object detection through layer-wise representations of images by deep neural networks, in Section 6.5
- since multiscale feature maps closer to the top of Figure 4 are smaller, but have larger receptive fields, they are suitable for detecting fewer, but larger objects
- in a nutshell, via its base network and several multiscale feature map blocks, single-shot multibox detection generates a varying number of anchor boxes with different sizes, and detects varying-size objects by predicting classes and offsets of these anchor boxes (thus the bounding boxes); thus, this is a multiscale object detection model

# 6.6 Single shot multibox detection

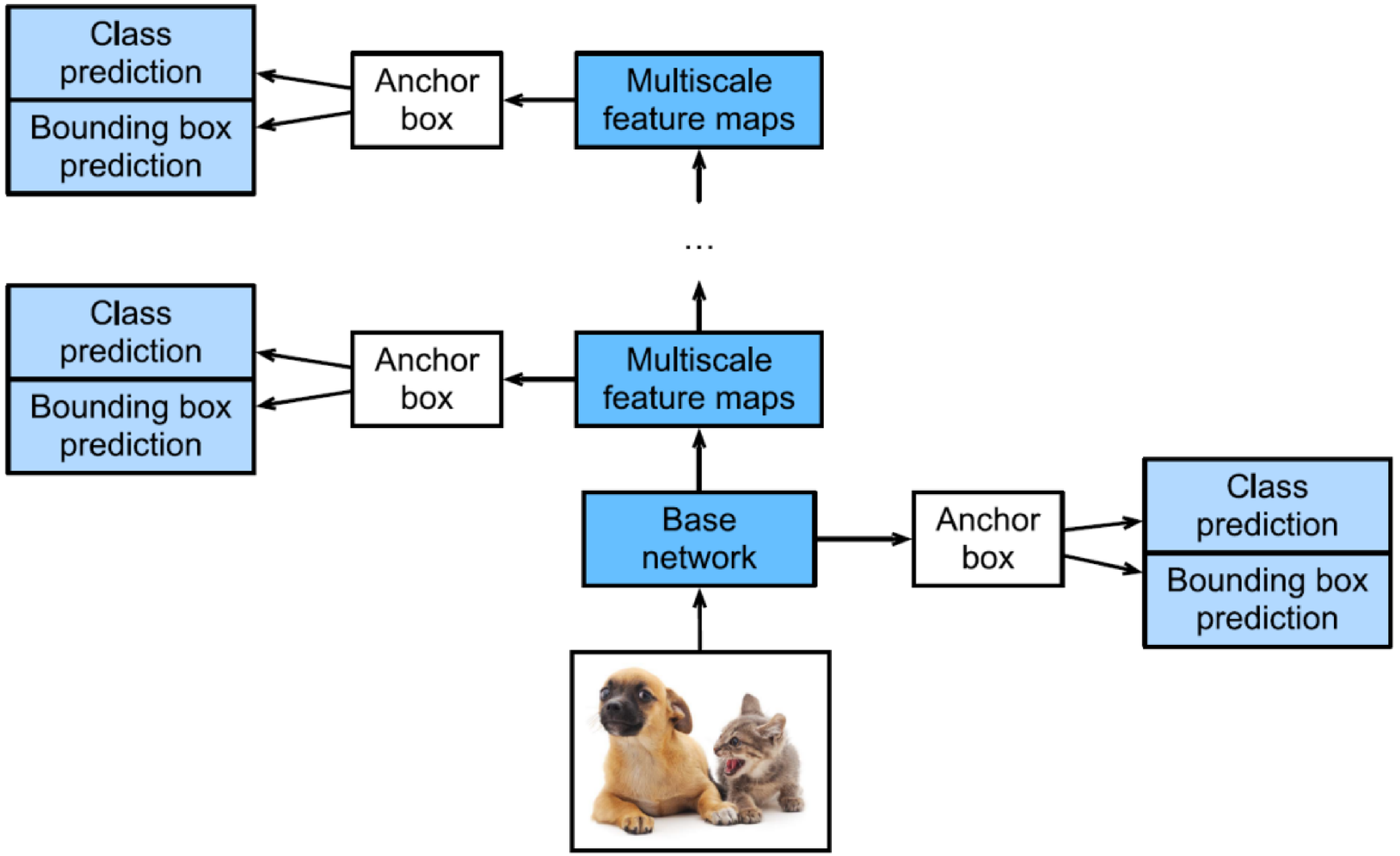



Figure 4: As a multiscale object detection model, single-shot multibox detection mainly consists of a base network followed by several multiscale feature map blocks.

# 6.6 Single shot multibox detection

- in the following, we will describe the details of different blocks in Figure 4
- to begin with, we discuss how to implement the class and bounding box prediction
- let the number of object classes be $q$; then, anchor boxes have $q + 1$ classes, where class 0 is background
- at some scale, suppose that the height and width of feature maps are $h$ and $w$, respectively
- when $a$ anchor boxes are generated with each spatial position of these feature maps as their center, a total of $hwa$ anchor boxes need to be classified
- this often makes classification with fully-connected layers infeasible, due to likely heavy parameterization costs
- single-shot multibox detection uses channels of convolutional layers to predict classes, to reduce model complexity

# 6.6 Single shot multibox detection

- specifically, the class prediction layer uses a convolutional layer, without altering the width or height of the feature maps
- in this way, there can be a one-to-one correspondence between outputs and inputs at the same spatial dimensions (width and height) of feature maps
- more concretely, channels of the output feature maps at any spatial position $(x, y)$ represent class predictions for all the anchor boxes centered on $(x, y)$ of the input feature maps
- to produce valid predictions, there must be $a(q + 1)$ output channels, where, for the same spatial position, the output channel with index $i(q + 1) + j$ represents the prediction of the class $j$ $(0 \leq j \leq q)$ for the anchor box $i$ $(0 \leq i < a)$

# 6.6 Single shot multibox detection

- we can define such a class prediction layer, to use a $3 \times 3$ convolutional layer with a padding of 1
- the width and height of the input and output of this convolutional layer remain unchanged
- the design of the bounding box prediction layer is similar to that of the class prediction layer
- the only difference lies in the number of outputs for each anchor box: here, we need to predict four offsets, rather than $q + 1$ classes
- as we mentioned, single-shot multibox detection uses multiscale feature maps to generate anchor boxes and predict their classes and offsets
- at different scales, the shapes of feature maps or the numbers of anchor boxes centered on the same unit may vary
- therefore, shapes of the prediction outputs at different scales may vary

# 6.6 Single shot multibox detection

- in order to detect objects at multiple scales, we define a downsampling block that halves the height and width of input feature maps
- in fact, this block applies the design of VGG blocks in Section 3.8
- more concretely, each downsampling block consists of two $3 \times 3$ convolutional layers with padding of 1, followed by a $2 \times 2$ maximum pooling layer with stride of 2
- as we know, $3 \times 3$ convolutional layers with padding of 1 do not change the shape of feature maps
- however, the subsequent $2 \times 2$ maximum pooling reduces the height and width of input feature maps by half
- for both input and output feature maps of this downsampling block, because $1 \times 2 + (3 - 1) + (3 - 1) = 6$, each unit in the output has a $6 \times 6$ receptive field on the input
- therefore, the downsampling block enlarges the receptive field of each unit in its output feature maps

# 6.6 Single shot multibox detection

- the base network block is then used to extract features from input images
- for simplicity, we construct a small base network consisting of three downsampling blocks that double the number of channels at each block
- given a $256 \times 256$ input image, this base network block outputs $32 \times 32$ feature maps ($256/2^3 = 32$)
- the complete single shot multibox detection model consists of five blocks
- the feature maps produced by each block are used for both (i) generating anchor boxes and (ii) predicting classes and offsets of these anchor boxes
- among these five blocks, the first one is the base network block, the second to the fourth are downsampling blocks, and the last block uses global maximum pooling to reduce both the height and width to 1
- technically, the second to the fifth blocks are all those multiscale feature map blocks in Figure 4

# 6.6 Single shot multibox detection

- different from the case of image classification tasks, the outputs of each block include (i) CNN feature maps **Y**, (ii) anchor boxes generated using **Y** at the current scale, and (iii) classes and offsets predicted (based on **Y**) for these anchor boxes
- recall that, in Figure 4, a multiscale feature map block that is closer to the top is for detecting larger objects; thus, it needs to generate larger anchor boxes
- the interval between 0.2 and 1.05 has to be split evenly into five sections, to determine the smaller scale values at the five blocks: 0.2, 0.37, 0.54, 0.71, and 0.88
- then, their larger scale values are given by $\sqrt{0.2 \times 0.37} = 0.272$, $\sqrt{0.37 \times 0.54} = 0.447$, and so on

# 6.6 Single shot multibox detection

- for example, let's say we apply the model on a mini-batch of $256 \times 256$ images
- as explained earlier in this section, the first block outputs $32 \times 32$ feature maps
- recall that the second to fourth downsampling blocks halve the height and width, and the fifth block uses global pooling
- since 4 anchor boxes are generated for each unit along spatial dimensions of feature maps, at all the five scales, a total of $(32^2 + 16^2 + 8^2 + 4^2 + 1) \times 4 = 5444$ anchor boxes are generated for each image
- now, we will explain how to train the single shot multibox detection model for object detection

# 6.6 Single shot multibox detection

- object detection has two types of losses
- the first loss concerns classes of anchor boxes: its computation can simply reuse the cross-entropy loss function that we used for image classification
- the second loss concerns offsets of positive (non-background) anchor boxes: this is a regression problem
- however, for this regression problem, here, we do not use the squared loss
- instead, we use the $\ell_1$ norm loss, the absolute value of the difference between the prediction and the ground-truth
- as discussed in Section 6.4, we filter out negative anchor boxes and illegal (padded) anchor boxes in the loss calculation
- in the end, we sum up the anchor box class loss and the anchor box offset loss, to obtain the loss function for the model

# 6.6 Single shot multibox detection

- we can use accuracy to evaluate the classification results
- due to the used $\ell_1$ norm loss for the offsets, we can use the *mean absolute error* to evaluate the predicted bounding boxes
- these prediction results are obtained from the generated anchor boxes and the predicted offsets for them
- when training the model, we need to generate multiscale anchor boxes and predict their classes and offsets in the forward propagation
- then, we label the classes and offsets of such generated anchor boxes based on the label information

# 6.6 Single shot multibox detection

- finally, we calculate the loss function using the predicted and labeled values of the classes and offsets
- during prediction, the goal is to detect all the objects of interest on the image
- the predicted bounding boxes are then obtained from the anchor boxes and their predicted offsets
- then, non-maximum suppression is used to remove similar predicted bounding boxes
- finally, we display all the predicted bounding boxes with confidence 0.9 or above, as the output

# 6.7 Region-based CNNs (R-CNNs)

- besides single shot multibox detection described in Section 6.6, *region-based CNNs* or regions with CNN features (*R-CNNs*) are also among many pioneering approaches of applying deep learning to object detection
- in this section, we will introduce the R-CNN and its series of improvements: the fast R-CNN, the faster R-CNN, and the mask R-CNN
- the *R-CNN* first extracts many (e.g., 2000) *region proposals* from the input image (e.g., anchor boxes can also be considered as region proposals), labeling their classes and bounding boxes (e.g., offsets)
- then, a CNN is used to perform forward propagation on each region proposal to extract its features
- next, features of each region proposal are used for predicting the class and bounding box of this region proposal

# 6.7 Region-based CNNs (R-CNNs)

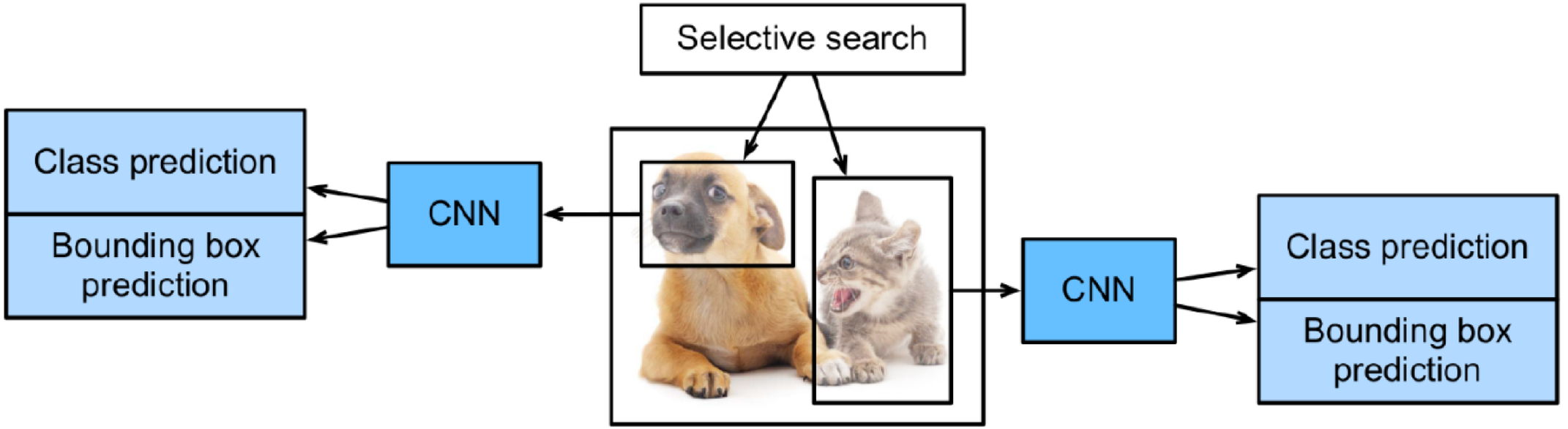



Figure 5: The R-CNN model.

# 6.7 Region-based CNNs (R-CNNs)

- Figure 5 shows the R-CNN model; more concretely, the R-CNN consists of the following four steps:
  1. Perform *selective search* to extract multiple high-quality region proposals on the input image. These proposed regions are usually selected at multiple scales, with different shapes and sizes. Each region proposal will be labeled with a class and a ground-truth bounding box.
  2. Choose a pretrained CNN, and truncate it before the output layer. Resize each region proposal to the input size required by the network, and output the extracted features for the region proposal through forward propagation.
  3. Take the extracted features and labeled class of each region proposal as an example. Train multiple support vector machines to classify objects, where each support vector machine individually determines whether the example contains a specific class.
  4. Take the extracted features and labeled bounding box of each region proposal as an example. Train a linear regression model to predict the ground-truth bounding box.

# 6.7 Region-based CNNs (R-CNNs)

- although the R-CNN model uses pretrained CNNs to effectively extract image features, it is slow
- imagine that we select thousands of region proposals from a single input image: this requires thousands of CNN forward propagations to perform object detection
- this massive computing load makes it infeasible to widely use R-CNNs in real-world applications
- the main performance bottleneck of an R-CNN lies in the independent CNN forward propagation for each region proposal, without sharing computation
- since these regions usually have overlaps, independent feature extractions lead to much repeated computation
- one of the major improvements of the *fast R-CNN* from the R-CNN is that the CNN forward propagation is only performed on the entire image

# 6.7 Region-based CNNs (R-CNNs)

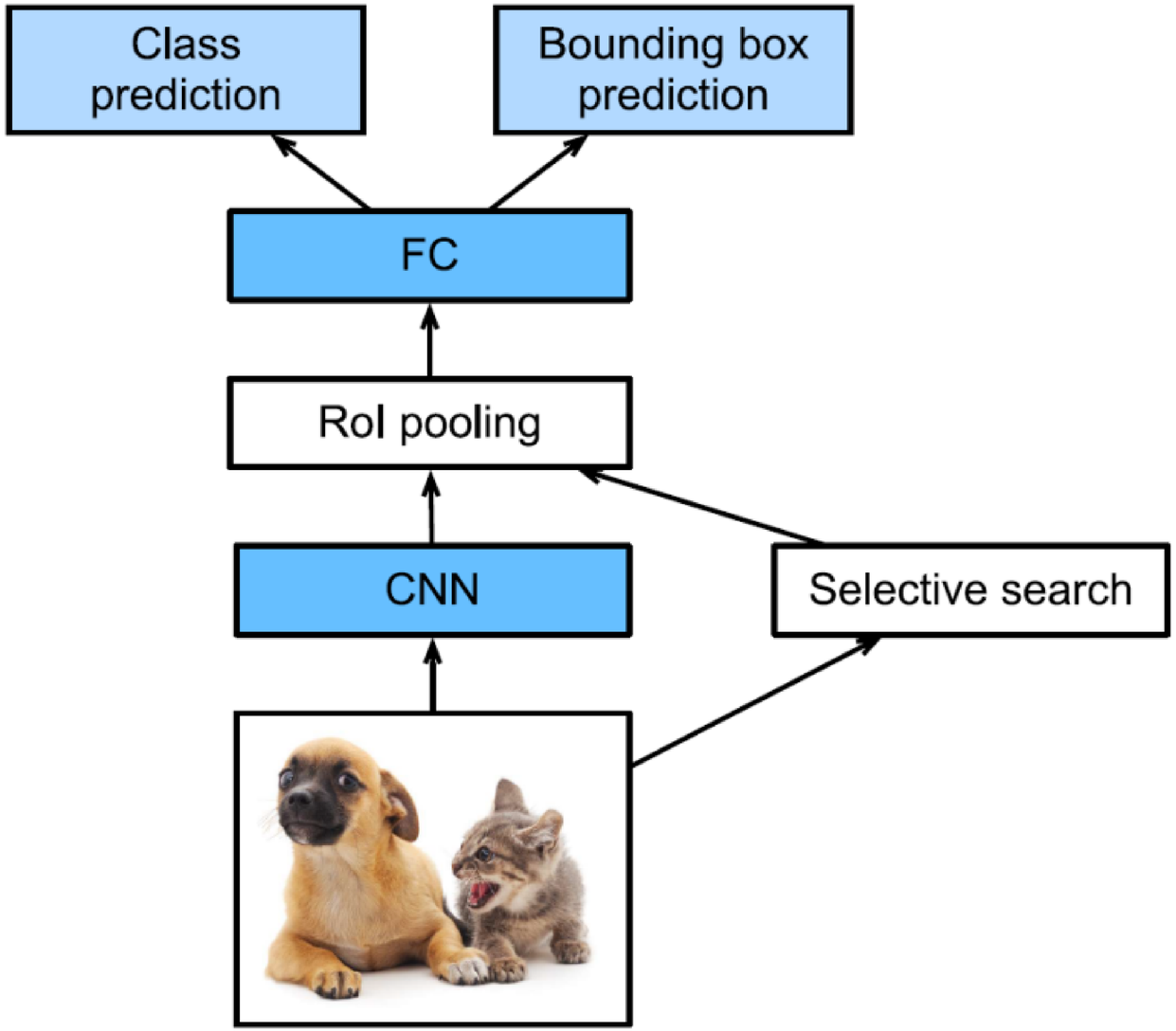



Figure 6: The fast R-CNN model.

# 6.7 Region-based CNNs (R-CNNs)

- Figure 6 describes the fast R-CNN model; its major computations are as follows:
  1. Compared with the R-CNN, in the fast R-CNN, the input of the CNN for feature extraction is the entire image, rather than individual region proposals. Moreover, this CNN is trainable. Given an input image, let the shape of the CNN output be $1 \times c \times h_1 \times w_1$.
  2. Suppose that selective search generates $n$ region proposals. These region proposals (of different shapes) mark regions of interest (of different shapes) on the CNN output. Then, these regions of interest further extract features of the same shape (say height $h_2$ and width $w_2$ are specified) in order to be easily concatenated. To achieve this, the fast R-CNN introduces the *region of interest (RoI) pooling* layer: the CNN output and region proposals are input into this layer, outputting concatenated features of shape $n \times c \times h_2 \times w_2$ that are further extracted for all the region proposals.
  3. Using a fully-connected layer, transform the concatenated features into an output of shape $n \times d$, where $d$ depends on the model design.
  4. Predict the class and bounding box for each of the $n$ region proposals. More concretely, in class and bounding box prediction, transform the fully-connected layer output into an output of shape $n \times q$ ($q$ is the number of classes) and an output of shape $n \times 4$, respectively. The class prediction uses softmax regression.

# 6.7 Region-based CNNs (R-CNNs)

- the region of interest pooling layer proposed in the fast R-CNN is different from the pooling layer introduced in Section 3.5
- in the pooling layer, we indirectly control the output shape by specifying sizes of the pooling window, padding, and stride
- in contrast, we can directly specify the output shape in the region of interest pooling layer
- for example, let us specify the output height and width for each region as $h_2$ and $w_2$, respectively
- for any region of interest window of shape $h \times w$, this window is divided into a $h_2 \times w_2$ grid of subwindows, where the shape of each subwindow is approximately $(h/h_2) \times (w/w_2)$

# 6.7 Region-based CNNs (R-CNNs)

- in practice, the height and width of any subwindow shall be rounded up, and the largest element shall be used as the output of the subwindow
- therefore, the region of interest pooling layer can extract features of the same shape, even when regions of interest have different shapes
- as an illustrative example, in Figure 7, the upper-left $3 \times 3$ region of interest is selected on a $4 \times 4$ input
- for this region of interest, we use a $2 \times 2$ region of interest pooling layer to obtain a $2 \times 2$ output
- note that each of the four divided subwindows contains elements 0, 1, 4, and 5 (5 is the maximum); 2 and 6 (6 is the maximum); 8 and 9 (9 is the maximum); and 10

# 6.7 Region-based CNNs (R-CNNs)

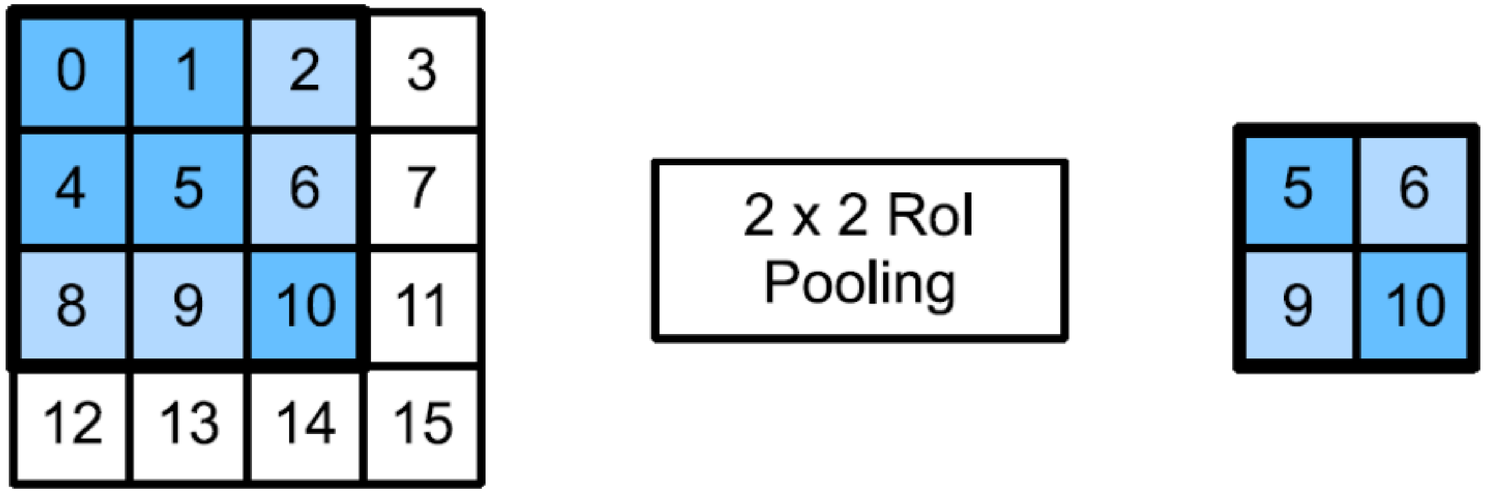



Figure 7: A $2 \times 2$ region of interest pooling layer.

# 6.7 Region-based CNNs (R-CNNs)

- to be more accurate in object detection, the fast R-CNN model usually has to generate a lot of region proposals in selective search
- to reduce region proposals without loss of accuracy, the *faster R-CNN* proposes to replace selective search with a *region proposal network*

# 6.7 Region-based CNNs (R-CNNs)

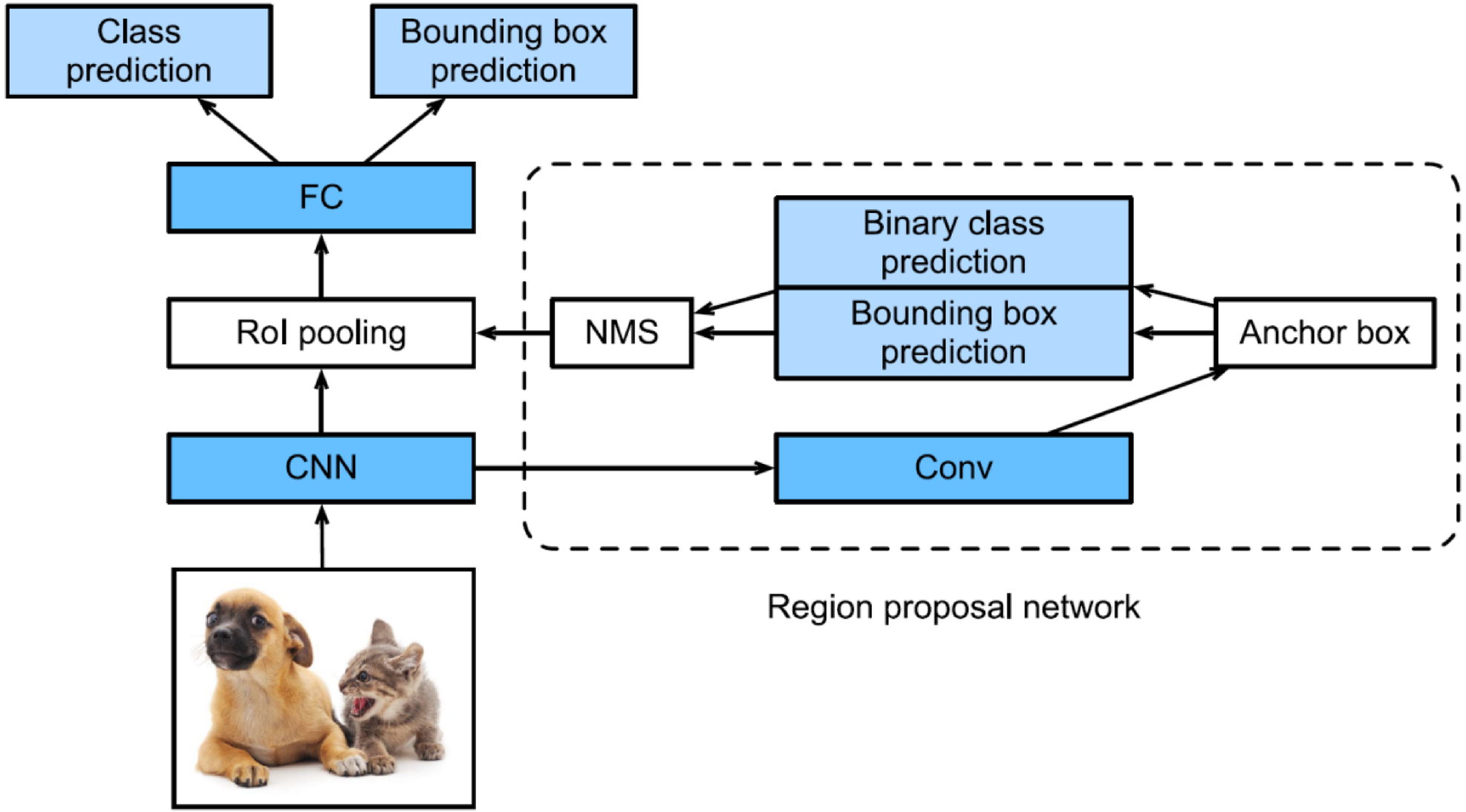



Figure 8: The faster R-CNN model.

# 6.7 Region-based CNNs (R-CNNs)

- Figure 8 shows the faster R-CNN model
- compared with the fast R-CNN, the faster R-CNN only changes the region proposal method from selective search to a region proposal network
- the rest of the model remains unchanged; the region proposal network works in the following steps:
  1. Use a $3 \times 3$ convolutional layer with padding of 1 to transform the CNN output to a new output with $c$ channels. In this way, each unit along the spatial dimensions of the CNN-extracted feature maps gets a new feature vector of length $c$.
  2. Centered on each pixel of the feature maps, generate multiple anchor boxes of different scales and aspect ratios and label them.
  3. Using the length-$c$ feature vector at the center of each anchor box, predict the binary class (background or objects) and bounding box for this anchor box.
  4. Consider those predicted bounding boxes whose predicted classes are objects. Remove overlapped results using non-maximum suppression. The remaining predicted bounding boxes for objects are the region proposals required by the region of interest pooling layer.

# 6.7 Region-based CNNs (R-CNNs)

- it is worth noting that, as part of the faster R-CNN model, the region proposal network is jointly trained with the rest of the model
- in other words, the objective function of the faster R-CNN includes not only the class and bounding box prediction in object detection, but also the binary class and bounding box prediction of anchor boxes in the region proposal network
- as a result of the end-to-end training, the region proposal network learns how to generate high-quality region proposals, so as to stay accurate in object detection, with a reduced number of region proposals that are learned from data
- in the training dataset, if pixel-level positions of objects are also labeled on images, the *mask R-CNN* can effectively leverage such detailed labels to further improve the accuracy of object detection

# 6.7 Region-based CNNs (R-CNNs)

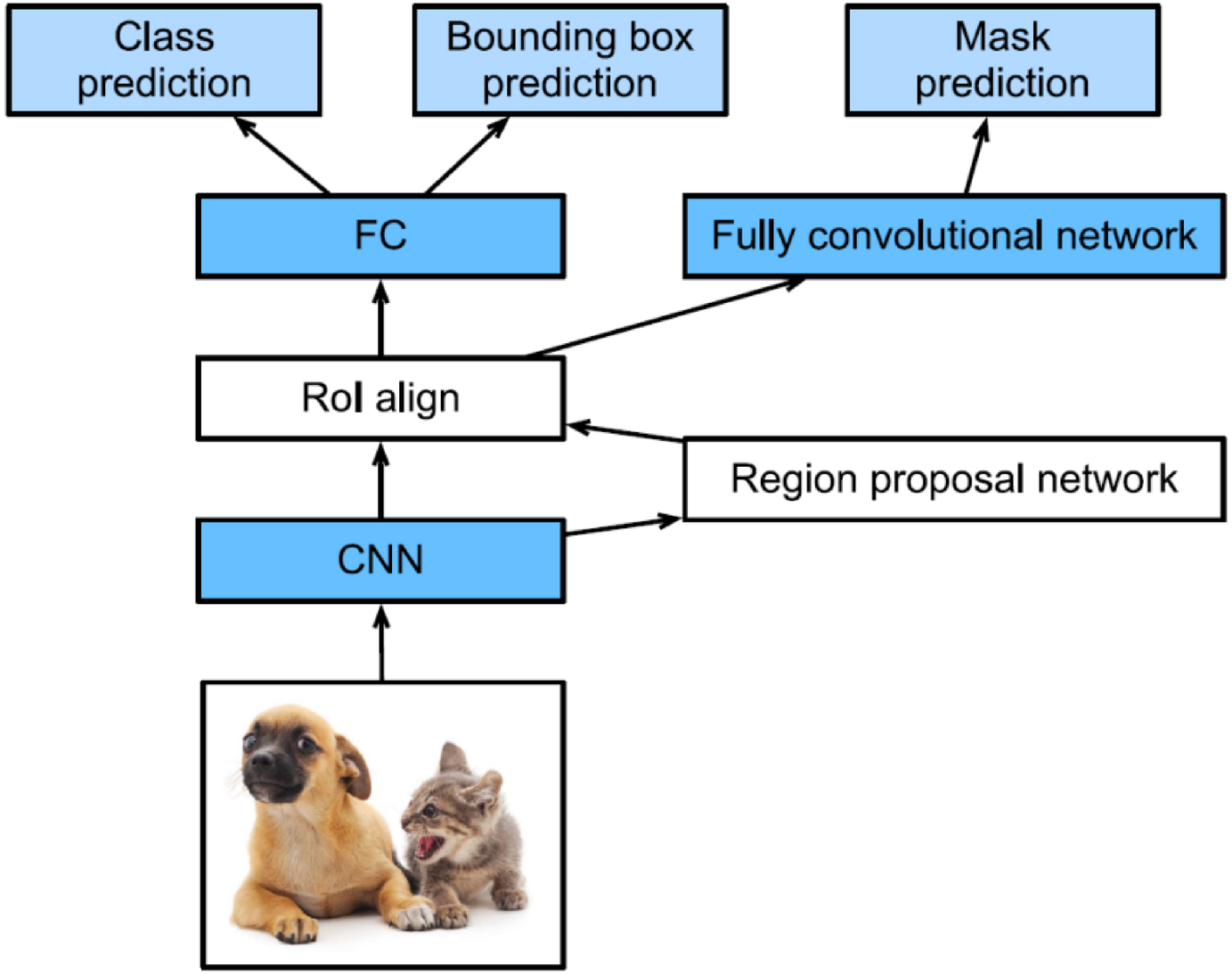



Figure 9: The mask R-CNN model.

# 6.7 Region-based CNNs (R-CNNs)

- as shown in Figure 9, the mask R-CNN is modified based on the faster R-CNN
- specifically, the mask R-CNN replaces the region of interest pooling layer with the *region of interest* (*RoI*) *alignment* layer
- this region of interest alignment layer uses bilinear interpolation to preserve the spatial information on the feature maps, which is more suitable for pixel-level prediction
- the output of this layer contains feature maps of the same shape, for all the regions of interest
- they are used to predict not only the class and bounding box for each region of interest, but also the pixel-level position of the object, through an additional fully convolutional network
- more details on using a fully convolutional network to predict pixel-level semantics of an image will be provided in the next chapter

Vă mulţumesc!